

Minimum Spanning Tree: Prim's Algorithm

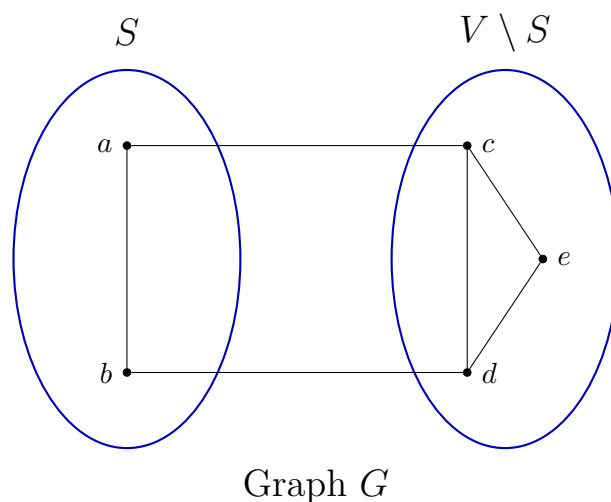
1 Introduction

The notion of a spanning tree is only applicable to **undirected graphs**.

1.1 Concept of a “Cut”

To define the algorithm, we first need to define the notion of a cut in an undirected graph.

A **cut** in an undirected graph $G = (V, E)$ is a partition of the vertex set V into two distinct parts (subsets).



For example, in the graph above, $\{a, b\}$ and $\{c, d, e\}$ form a cut in the graph G . There could be many other cuts in a graph. For instance, we could have only one vertex on one side, say $\{c\}$, and the other 4 vertices on the other side $\{a, b, d, e\}$.

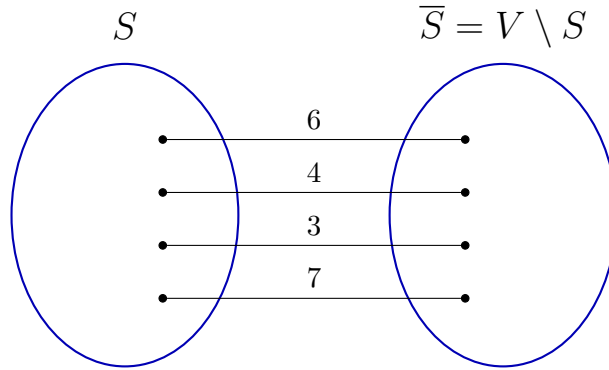
How many cuts can there be in a graph? It is $2^{n-1} - 1$. How did we get this value? Every vertex has two choices: either it can be in the left part or in the right part (2^n possibilities). Since a cut divides the graph into two parts, we divide by 2 to avoid repeating symmetric possibilities (e.g., partitioning into A and B is the same as B and A), yielding 2^{n-1} . Finally, we subtract 1 to exclude the trivial case where all vertices are on one side (an empty partition).

Cut Edges: A cut edge is an edge which has one endpoint on one side of the partition and the other endpoint on the other side.

2 The Cut Property

We need this notion to understand how to design the algorithm. Suppose we take a cut of a given graph $G = (V, E)$ into sets S and $V \setminus S$.

Let us look at all the edges in the cut and their lengths. To simplify the arguments and presentation, we will assume that **all edge lengths are distinct**. When edge lengths are distinct, the graph has a unique Minimum Spanning Tree (MST).



Claim: For any cut $(S, \bar{S})$, the minimum weight edge in the cut **must** belong to the MST.

Note: It is not necessary that *only* the minimum edge will be part of the MST. There could be more than one edge in the cut that appears in the spanning tree, but we are absolutely sure that the minimum edge must belong to the MST.

2.1 Proof of the Claim

We will prove this by contradiction. Let T be an MST which **does not** contain the minimum length edge e in the cut.

1. Add edge e to the tree T . Since T was a spanning tree, adding a new edge e will inevitably form a cycle, say C .
2. Because the endpoints of edge e are on opposite sides of the cut (one in S , one in $V \setminus S$), the cycle C must cross the cut at least one more time to return to the starting side. Therefore, the cycle C contains at least one other edge from the cut, let's call it e' .
3. Since e is the minimum edge in the cut, the length of e' must be strictly greater than the length of e .
4. By removing this edge e' from the graph $T \cup \{e\}$, we break the cycle and form a new spanning tree T' .
5. The total cost of T' is smaller than T because we replaced a heavier edge (e') with a lighter edge (e).

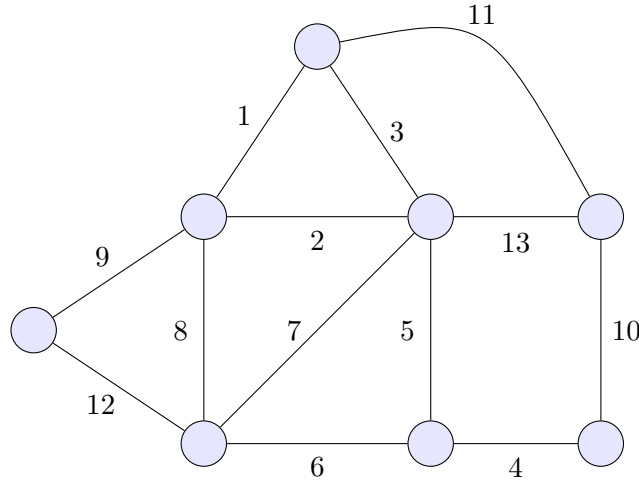
This contradicts our initial assumption that T was a Minimum Spanning Tree. Therefore, the minimum edge e in the cut must be part of the MST.

3 Prim's Algorithm

Prim's algorithm uses this simple fact. It builds the MST by maintaining a cut at every step and always picking the minimum edge across the cut. Since we have proved the claim, picking the minimum edge guarantees we are building a valid MST.

3.1 Algorithm Trace

Let's illustrate Prim's algorithm on the following graph. We start from an arbitrary vertex (which we'll call the root). In the beginning, the root is on one side of the cut (S), and everyone else is on the other side ($\bar{S}$).



Step-by-Step Execution:

1. **Start at top node.** The cut separates the top node from all others. The edges in the cut are $\{1, 3, 11\}$. The minimum edge is **1**. We include it in the MST.
2. The cut now encompasses the top node and the left-mid node. The edges crossing the cut are $\{3, 11, 2, 9, 8\}$. The smallest among them is **2**. We include it.
3. The cut encompasses the top, left-mid, and right-mid nodes. Edge 3 is now internal and ignored. The cut edges are $\{11, 9, 8, 7, 5, 13\}$. The smallest is **5**. We include it.
4. The cut expands to include the bottom-mid node. The new cut edges are $\{11, 9, 8, 7, 13, 6, 4\}$. The smallest is **4**. We include it.
5. We include the bottom-right node. Cut edges: $\{11, 9, 8, 7, 13, 6, 10\}$. Smallest is **6**.
6. We include the bottom-left node. Cut edges: $\{11, 9, 13, 10, 12\}$. (Edges 7 and 8 are now internal to S). The smallest is **9**.
7. We include the far-left node. Cut edges: $\{11, 13, 10\}$. The smallest is **10**.
8. We include the far-right node. All vertices are now in S . The algorithm terminates.

4 Implementation and Complexity

How do we efficiently implement this algorithm? At any point, we have a set S containing the root and vertices reached so far. The remaining vertices are in $V \setminus S$. We can maintain a simple array to keep track of which vertices are in S (1 if in S , 0 if in $V \setminus S$).

At every step, we have to look at all edges in the cut and find the minimum.

- **Naive Approach:** Look at all vertices in S , check all their adjacent edges, see if the other endpoint is in $V \setminus S$, and find the minimum. This requires searching extensively and takes $\mathcal{O}(nm)$ time, where n is vertices and m is edges.
- **Clever Approach (Using a Min-Heap):** Instead of searching blindly, we can maintain a Min-Heap. The heap will store all the edges currently in the cut.

4.1 Using a Min-Heap

A Min-Heap allows us to:

- Find the minimum element in $\mathcal{O}(1)$ time.
- Insert an element or delete the minimum element in $\mathcal{O}(\log n)$ time.

When a vertex v is moved from $V \setminus S$ into S : 1. We **delete** the edges from the heap that just became internal to S (i.e., edges connecting v to other vertices already in S). 2. We **insert** the new edges into the heap that now cross the cut (i.e., edges connecting v to vertices still in $V \setminus S$).

Time Complexity Analysis: Each edge is inserted into the heap at most once and deleted at most once. For a vertex v with degree $d(v)$, we spend $\mathcal{O}(d(v) \log n)$ time processing its adjacent edges. Summing this over all vertices, the total time spent on heap operations is:

$$\sum_{v \in V} d(v) \log n = 2m \log n = \mathcal{O}(m \log n)$$

where m is the total number of edges in the graph.

5 Pseudocode

Below is the pseudocode representing the Min-Heap implementation of Prim's Algorithm.

Algorithm 1 Prim's Algorithm (Min-Heap based)

```

1:  $S \leftarrow$  array of size  $|V|$  initialized to 0  $\triangleright S[v] = 1$  if  $v \in S$ , else 0
2:  $H \leftarrow$  empty Min-Heap
3:  $root \leftarrow$  an arbitrary starting vertex
4:  $S[root] \leftarrow 1$ 
5: for each edge  $e = (root, v)$  adjacent to  $root$  do
6:   Insert  $e$  into  $H$ 
7: end for
8: while  $H$  is not empty do
9:    $e = (u, v) \leftarrow \text{Extract-Min}(H)$   $\triangleright$  Let  $u$  be the endpoint in  $S$ ,  $v$  in  $\bar{S}$ 
10:  if  $S[v] == 0$  then  $\triangleright$  If  $v$  is not yet in the MST
11:     $S[v] \leftarrow 1$   $\triangleright$  Add  $v$  to set  $S$ 
12:    Add edge  $e$  to MST
13:    for each edge  $e' = (v, w)$  adjacent to  $v$  do
14:      if  $S[w] == 0$  then  $\triangleright$  If  $w$  is on the other side of the cut
15:        Insert  $e'$  into  $H$ 
16:      end if
17:    end for
18:  end if
19: end while

```
